Bhoomi Annamdevara
24BCI0130

COMPILER DESIGN LAB
Lab Assignment - 05

Three-Address Code (TAC) Generation for Arithmetic Assignment Statements

Aim:

To implement a program in C that takes an assignment statement containing an arithmetic expression as input and generates its equivalent Three-Address Code (TAC). The program parses variable names, replaces them with standardized identifier tokens (id1, id2, etc.), and processes operator precedence and associativity rules to produce intermediate temporary instructions.

Algorithm:

1. Start the program.
2. Accept an assignment statement of the form lhs = rhs; from the user.
3. Split the statement at the '=' symbol into the left-hand side (lhs) and the right-hand side (rhs), and remove the trailing semicolon from the rhs.
4. Tokenize the rhs into operands and the operators (+, -, *, /) between them.
5. For each operand, check whether it is a numerical constant or a user-defined variable name.
6. Convert every user-defined variable name into a standardized identifier of the form idN, where N is a unique sequence number starting from 1, assigned in order of first occurrence. Reuse the same idN for repeated occurrences of the same variable. Leave numerical constants unchanged.
7. Print the converted assignment statement in Standard Format using the standardized identifiers.
8. Resolve all multiplication and division operations first, left to right, generating a temporary variable (t1, t2, ...) for each and printing the corresponding Three-Address Code line.
9. Resolve the remaining addition and subtraction operations left to right in the same way, generating further temporary variables.
10. Assign the final temporary result to the standardized left-hand side identifier and print this as the last Three-Address Code line.
11. Stop the program.

Program:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

#define MAX 200

char input[MAX];
char lhs[50];
char rhs[MAX];

char varNames[50][50];
char varIds[50][10];
int varCount = 0;
```

```
char tokens[100][50];   /* operands (identifiers/constants), standardized */
char ops[100];          /* operators between tokens */
int tokenCount = 0;

int tempCount = 0;

/* Return standardized id for a variable name, assigning new id if first seen.
   Numeric constants are returned unchanged. */
void standardize(char name[], char out[])
{
    /* check if it's a pure number (int or float) */
    int isNum = 1;
    for(int i = 0; name[i] != '\0'; i++)
    {
        if(!isdigit(name[i]) && name[i] != '.')
        {
            isNum = 0;
            break;
        }
    }
    if(isNum)
    {
        strcpy(out, name);
        return;
    }

    /* look up existing mapping */
    for(int i = 0; i < varCount; i++)
    {
        if(strcmp(varNames[i], name) == 0)
        {
            strcpy(out, varIds[i]);
            return;
        }
    }
    /* new variable */
    strcpy(varNames[varCount], name);
    sprintf(varIds[varCount], "id%d", varCount + 1);
    strcpy(out, varIds[varCount]);
    varCount++;
}

int main()
{
    printf("Enter assignment statement:\n");
    fgets(input, sizeof(input), stdin);
    input[strcspn(input, "\n")] = '\0';

    char *eq = strchr(input, '=');
    if(eq == NULL)
    {
        printf("Invalid statement, no '=' found\n");
        return 0;
    }
    int lhsLen = eq - input;
    strncpy(lhs, input, lhsLen);
    lhs[lhsLen] = '\0';
    strcpy(rhs, eq + 1);

    int rl = strlen(rhs);
    if(rl > 0 && rhs[rl-1] == ';')
        rhs[rl-1] = '\0';

    /* Tokenize rhs into operands and operators (+, -, *, /) */
    int i = 0, j = 0;
```

```
char tok[50];
int len = strlen(rhs);

while(i <= len)
{
    if(rhs[i]=='+' || rhs[i]=='-' || rhs[i]=='*' || rhs[i]=='/' || rhs[i]=='\0')
    {
        tok[j] = '\0';
        if(j > 0)
        {
            strcpy(tokens[tokenCount], tok);
            tokenCount++;
        }
        if(rhs[i] != '\0')
            ops[tokenCount - 1] = rhs[i]; /* operator following this token */
        j = 0;
    }
    else
    {
        tok[j++] = rhs[i];
    }
    i++;
}

/* Standardize lhs */
char stdLhs[10];
standardize(lhs, stdLhs);

/* Standardize each operand token */
char stdTokens[100][50];
for(int k = 0; k < tokenCount; k++)
    standardize(tokens[k], stdTokens[k]);

/* Print standard format */
printf("\nStandard format:\n");
printf("%s=", stdLhs);
for(int k = 0; k < tokenCount; k++)
{
    printf("%s", stdTokens[k]);
    if(k < tokenCount - 1)
        printf("%c", ops[k]);
}
printf(";\n");

/* Generate Three Address Code using two-pass precedence resolution:
   Pass 1: resolve all * and / left to right, collapsing into temps
   Pass 2: resolve remaining + and - left to right */

char curTok[100][50];
char curOps[100];
int curCount = tokenCount;
for(int k = 0; k < tokenCount; k++)
    strcpy(curTok[k], stdTokens[k]);
for(int k = 0; k < tokenCount - 1; k++)
    curOps[k] = ops[k];

printf("\nThree Address Code:\n");

/* Pass 1: * and / */
int k = 0;
while(k < curCount - 1)
{
    if(curOps[k] == '*' || curOps[k] == '/')
    {
        char t[10];
```

```
            sprintf(t, "t%d", ++tempCount);
            printf("%s = %s %c %s;\n", t, curTok[k], curOps[k], curTok[k+1]);

            /* replace curTok[k] and curTok[k+1] with t, shift left */
            strcpy(curTok[k], t);
            for(int m = k+1; m < curCount - 1; m++)
                strcpy(curTok[m], curTok[m+1]);
            for(int m = k; m < curCount - 2; m++)
                curOps[m] = curOps[m+1];
            curCount--;
            /* stay at same k to re-check (in case next op is also * or /) */
        }
        else
        {
            k++;
        }
    }

    /* Pass 2: + and - (left to right) */
    k = 0;
    while(curCount > 1)
    {
        char t[10];
        sprintf(t, "t%d", ++tempCount);
        printf("%s = %s %c %s;\n", t, curTok[0], curOps[0], curTok[1]);

        strcpy(curTok[0], t);
        for(int m = 1; m < curCount - 1; m++)
            strcpy(curTok[m], curTok[m+1]);
        for(int m = 0; m < curCount - 2; m++)
            curOps[m] = curOps[m+1];
        curCount--;
    }

    printf("%s = %s;\n", stdLhs, curTok[0]);

    return 0;
}
```

Input/Output Screenshots:



main.c

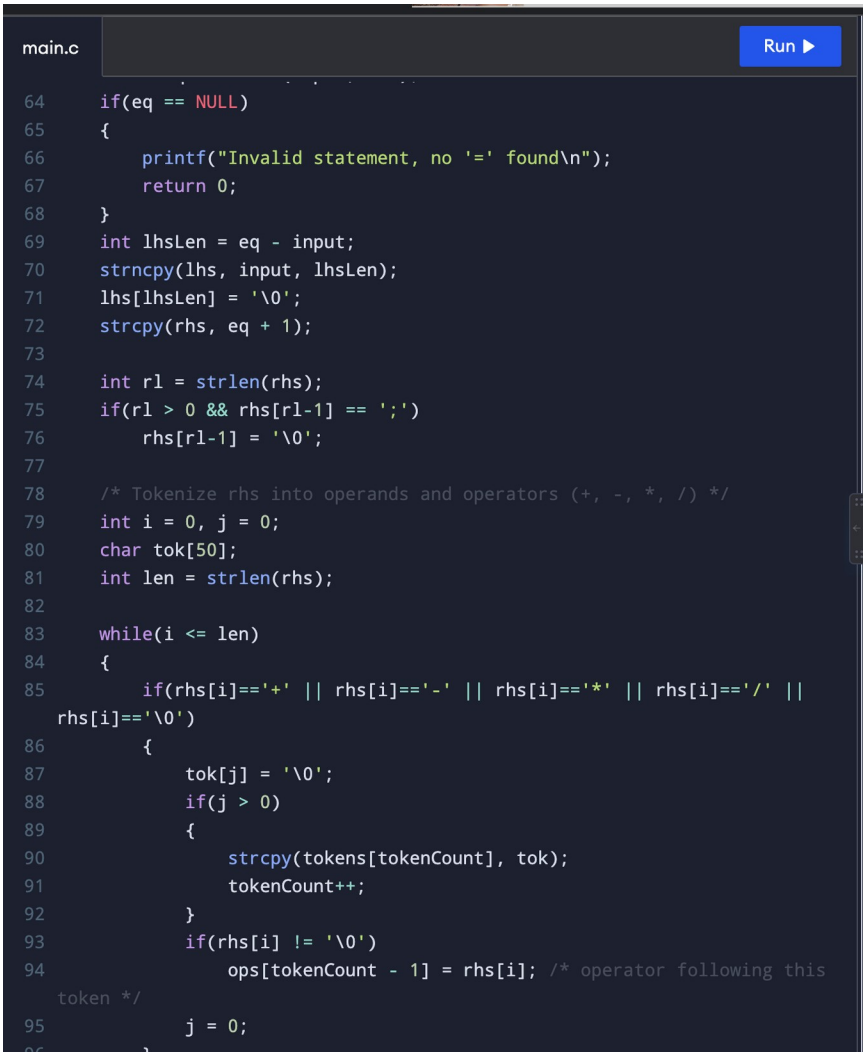
Run ▶

```
            isNum = 0;
            break;
        }
    }
    if(isNum)
    {
        strcpy(out, name);
        return;
    }

    /* look up existing mapping */
    for(int i = 0; i < varCount; i++)
    {
        if(strcmp(varNames[i], name) == 0)
        {
            strcpy(out, varIds[i]);
            return;
        }
    }
    /* new variable */
    strcpy(varNames[varCount], name);
    sprintf(varIds[varCount], "id%d", varCount + 1);
    strcpy(out, varIds[varCount]);
    varCount++;
}

int main()
{
    printf("Enter assignment statement:\n");
    fgets(input, sizeof(input), stdin);
    input[strcspn(input, "\n")] = '\0';

    char *eq = strchr(input, '=');
    if(eq == NULL)
    {
```


main.c
Run
if(eq == NULL)
{
printf("Invalid statement, no '=' found\n");
return 0;
}
int lhsLen = eq - input;
strncpy(lhs, input, lhsLen);
lhs[lhsLen] = '\0';
strcpy(rhs, eq + 1);
int rl = strlen(rhs);
if(rl > 0 && rhs[rl-1] == ';')
rhs[rl-1] = '\0';
/* Tokenize rhs into operands and operators (+, -, *, /) */
int i = 0, j = 0;
char tok[50];
int len = strlen(rhs);
while(i <= len)
{
if(rhs[i]=='+' || rhs[i]=='-' || rhs[i]=='*' || rhs[i]=='/' || rhs[i]=='\0')
{
tok[j] = '\0';
if(j > 0)
{
strcpy(tokens[tokenCount], tok);
tokenCount++;
}
if(rhs[i] != '\0')
ops[tokenCount - 1] = rhs[i]; /* operator following this token */
j = 0;

main.c

Run ▶

```
            if(rhs[i] != '\0')
                ops[tokenCount - 1] = rhs[i]; /* operator following this token */
            j = 0;
        }
        else
        {
            tok[j++] = rhs[i];
        }
        i++;
    }

    /* Standardize lhs */
    char stdLhs[10];
    standardize(lhs, stdLhs);

    /* Standardize each operand token */
    char stdTokens[100][50];
    for(int k = 0; k < tokenCount; k++)
        standardize(tokens[k], stdTokens[k]);

    /* Print standard format */
    printf("\nStandard format:\n");
    printf("%s=", stdLhs);
    for(int k = 0; k < tokenCount; k++)
    {
        printf("%s", stdTokens[k]);
        if(k < tokenCount - 1)
            printf("%c", ops[k]);
    }
    printf(";\n");

    /* Generate Three Address Code using two-pass precedence resolution:
       Pass 1: resolve all * and / left to right, collapsing into temps
```

main.c

Run ▶

```
    /* Generate Three Address Code using two-pass precedence resolution:
       Pass 1: resolve all * and / left to right, collapsing into temps
       Pass 2: resolve remaining + and - left to right */

    char curTok[100][50];
    char curOps[100];
    int curCount = tokenCount;
    for(int k = 0; k < tokenCount; k++)
        strcpy(curTok[k], stdTokens[k]);
    for(int k = 0; k < tokenCount - 1; k++)
        curOps[k] = ops[k];

    printf("\nThree Address Code:\n");

    /* Pass 1: * and / */
    int k = 0;
    while(k < curCount - 1)
    {
        if(curOps[k] == '*' || curOps[k] == '/')
        {
            char t[10];
            sprintf(t, "t%d", ++tempCount);
            printf("%s = %s %c %s;\n", t, curTok[k], curOps[k],
curTok[k+1]);

            /* replace curTok[k] and curTok[k+1] with t, shift left */
            strcpy(curTok[k], t);
            for(int m = k+1; m < curCount - 1; m++)
                strcpy(curTok[m], curTok[m+1]);
            for(int m = k; m < curCount - 2; m++)
                curOps[m] = curOps[m+1];
            curCount--;
            /* stay at same k to re-check (in case next op is also * or
/) */
```

main.c

Run ▶

```
            strcpy(curTok[k], t);
            for(int m = k+1; m < curCount - 1; m++)
                strcpy(curTok[m], curTok[m+1]);
            for(int m = k; m < curCount - 2; m++)
                curOps[m] = curOps[m+1];
            curCount--;
            /* stay at same k to re-check (in case next op is also * or
    /) */
        }
        else
        {
            k++;
        }
    }

    /* Pass 2: + and - (left to right) */
    k = 0;
    while(curCount > 1)
    {
        char t[10];
        sprintf(t, "t%d", ++tempCount);
        printf("%s = %s %c %s;\n", t, curTok[0], curOps[0], curTok[1]);

        strcpy(curTok[0], t);
        for(int m = 1; m < curCount - 1; m++)
            strcpy(curTok[m], curTok[m+1]);
        for(int m = 0; m < curCount - 2; m++)
            curOps[m] = curOps[m+1];
        curCount--;
    }

    printf("%s = %s;\n", stdLhs, curTok[0]);

    return 0;
}
```

Output

```
Enter assignment statement:
Isla=Test+Compiler*Design/Lab-22;

Standard format:
id1=id2+id3*id4/id5-22;

Three Address Code:
t1 = id3 * id4;
t2 = t1 / id5;
t3 = id2 + t2;
t4 = t3 - 22;
id1 = t4;


=== Code Execution Successful ===
```

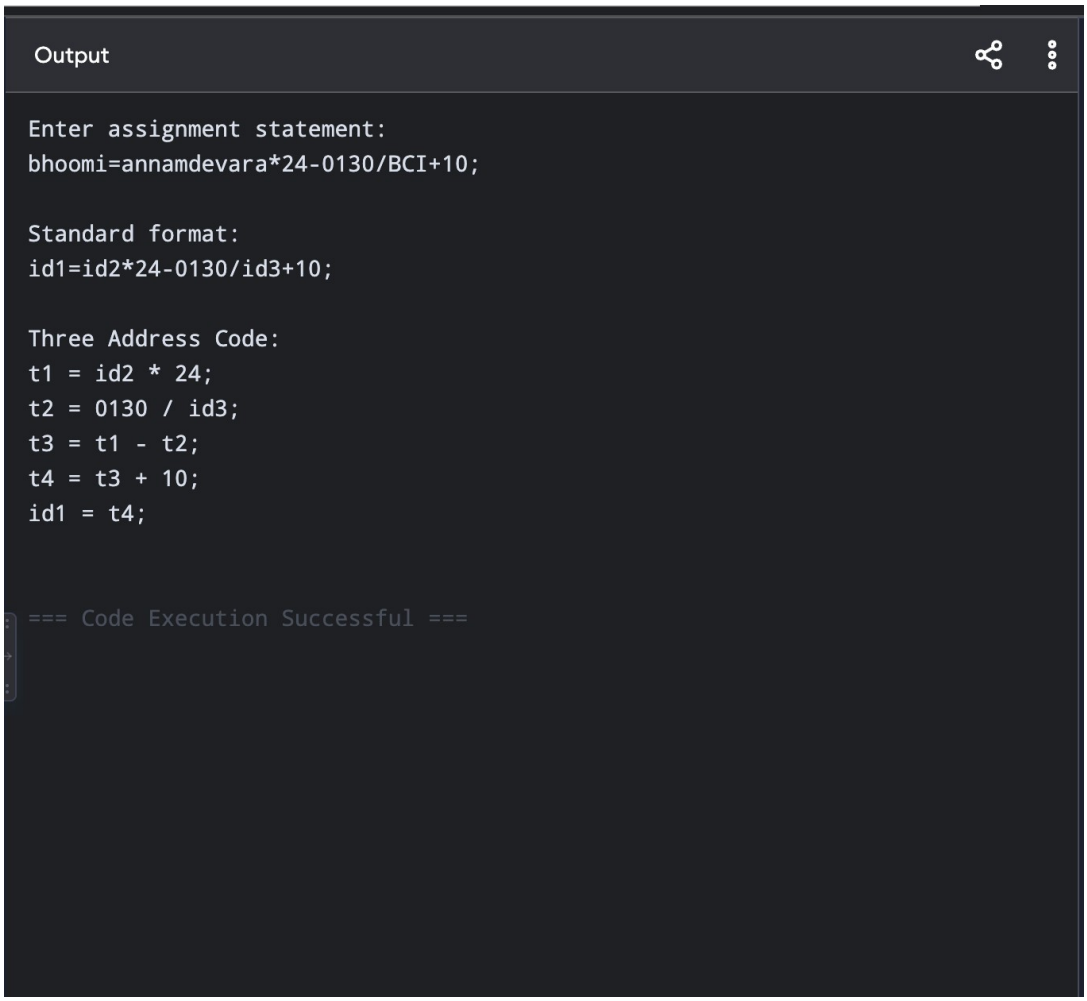
Output

```
Enter assignment statement:
a=b*c/a+d-e+BCE0022;

Standard format:
id1=id2*id3/id1+id4-id5+id6;

Three Address Code:
t1 = id2 * id3;
t2 = t1 / id1;
t3 = t2 + id4;
t4 = t3 - id5;
t5 = t4 + id6;
id1 = t5;


=== Code Execution Successful ===
```



Result:

Thus, the program to generate Three-Address Code (TAC) for arithmetic assignment statements was successfully developed. It correctly maps variable names to standardized identifiers, applies operator precedence and left-to-right associativity, and produces the intermediate temporary instructions along with the final assignment.

Translation of Three Address Code to Assembly Code

Aim:

To implement a program in C that translates a given Three-Address Code statement of the form x = y op z into its equivalent Assembly Code, using the register R1, and correctly distinguishing floating-point operations from integer operations.

Algorithm:

1. Start the program.
2. Accept a Three-Address Code statement of the form x = y op z; from the user, and remove the trailing semicolon.
3. Split the statement at the '=' symbol to extract x, and store the remainder as the right-hand side.
4. Scan the right-hand side to locate the operator (+, -, *, or /) and split it into the two operands y and z.
5. Check whether either operand y or z contains a decimal point, to determine whether the operation is a floating-point operation.
6. Select the base opcode corresponding to the operator: ADD for '+', SUB for '-', MUL for '*', and DIV for '/'.
7. If the operation is a floating-point operation, append the suffix 'F' to the opcode and use MOVF instead of MOV for all data movement instructions.
8. Print MOV z, R1 (or MOVF z, R1 for float), prefixing z with '#' if it is a numeric constant.
9. Print the opcode followed by y, R1, prefixing y with '#' if it is a numeric constant.
10. Print MOV R1, x (or MOVF R1, x for float) to store the result back into x.
11. Stop the program.

Program:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

#define MAX 100

char input[MAX];
char x[50], y[50], z[50];
char op;

int isFloatOperand(char s[])
{
    for(int i = 0; s[i] != '\0'; i++)
    {
        if(s[i] == '.')
            return 1;
    }
    return 0;
}

int isNumber(char s[])
{
    if(s[0] == '\0')
        return 0;
    for(int i = 0; s[i] != '\0'; i++)
```

```
    {
        if(!isdigit(s[i]) && s[i] != '.')
            return 0;
    }
    return 1;
}

void printOperand(char s[])
{
    if(isNumber(s))
        printf("#%s", s);
    else
        printf("%s", s);
}

int main()
{
    printf("Enter the Three Address Code : ");
    fgets(input, sizeof(input), stdin);
    input[strcspn(input, "\n")] = '\0';

    int rl = strlen(input);
    if(rl > 0 && input[rl-1] == ';')
        input[rl-1] = '\0';

    /* Parse x = y op z */
    char *eq = strchr(input, '=');
    if(eq == NULL)
    {
        printf("Invalid Three Address Code\n");
        return 0;
    }
    int xLen = eq - input;
    strncpy(x, input, xLen);
    x[xLen] = '\0';

    char rhs[80];
    strcpy(rhs, eq + 1);

    /* find operator among + - * / */
    int i = 0, opPos = -1;
    int len = strlen(rhs);
    for(i = 1; i < len; i++) /* start at 1 to skip a possible leading '-' */
    {
        if(rhs[i]=='+' || rhs[i]=='-' || rhs[i]=='*' || rhs[i]=='/')
        {
            opPos = i;
            break;
        }
    }

    if(opPos == -1)
    {
        printf("Invalid Three Address Code\n");
        return 0;
    }

    op = rhs[opPos];
    strncpy(y, rhs, opPos);
    y[opPos] = '\0';
    strcpy(z, rhs + opPos + 1);

    int isFloat = isFloatOperand(y) || isFloatOperand(z);

    char opcode[10];
```

```
    if(op == '+')
        strcpy(opcode, "ADD");
    else if(op == '-')
        strcpy(opcode, "SUB");
    else if(op == '*')
        strcpy(opcode, "MUL");
    else if(op == '/')
        strcpy(opcode, "DIV");

    if(isFloat)
        strcat(opcode, "F");

    char movOp[10];
    strcpy(movOp, isFloat ? "MOVF" : "MOV");

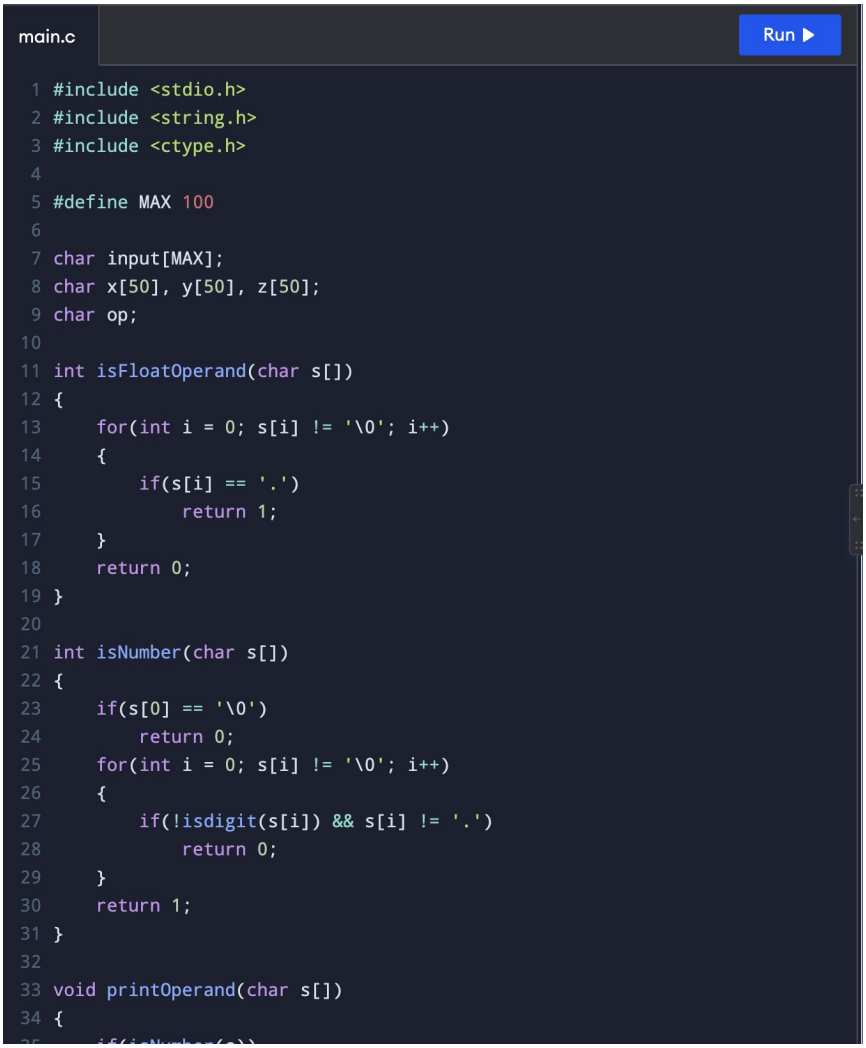
    printf("\n%s ", movOp);
    printOperand(z);
    printf(",R1\n");

    printf("%s ", opcode);
    printOperand(y);
    printf(",R1\n");

    printf("%s R1,%s\n", movOp, x);

    return 0;
}
```

Input/Output Screenshots:



main.c

Run ▶

```
{
    if(isNumber(s))
        printf("#%s", s);
    else
        printf("%s", s);
}

int main()
{
    printf("Enter the Three Address Code : ");
    fgets(input, sizeof(input), stdin);
    input[strcspn(input, "\n")] = '\0';

    int rl = strlen(input);
    if(rl > 0 && input[rl-1] == ';')
        input[rl-1] = '\0';

    /* Parse x = y op z */
    char *eq = strchr(input, '=');
    if(eq == NULL)
    {
        printf("Invalid Three Address Code\n");
        return 0;
    }
    int xLen = eq - input;
    strncpy(x, input, xLen);
    x[xLen] = '\0';

    char rhs[80];
    strcpy(rhs, eq + 1);

    /* find operator among + - * / */
    int i = 0, opPos = -1;
    int len = strlen(rhs);
    for(i = 1; i < len; i++) /* start at 1 to skip a possible leading '-'
```

main.c

Run ▶

```
    /* find operator among + - * / */
    int i = 0, opPos = -1;
    int len = strlen(rhs);
    for(i = 1; i < len; i++) /* start at 1 to skip a possible leading '-'
*/
    {
        if(rhs[i]=='+' || rhs[i]=='-' || rhs[i]=='*' || rhs[i]=='/')
        {
            opPos = i;
            break;
        }
    }

    if(opPos == -1)
    {
        printf("Invalid Three Address Code\n");
        return 0;
    }

    op = rhs[opPos];
    strncpy(y, rhs, opPos);
    y[opPos] = '\0';
    strcpy(z, rhs + opPos + 1);

    int isFloat = isFloatOperand(y) || isFloatOperand(z);

    char opcode[10];
    if(op == '+')
        strcpy(opcode, "ADD");
    else if(op == '-')
        strcpy(opcode, "SUB");
    else if(op == '*')
        strcpy(opcode, "MUL");
    else if(op == '/')
        strcpy(opcode, "DIV");
```

main.c

Run ▶

```
    strncpy(y, rhs, opPos);
    y[opPos] = '\0';
    strcpy(z, rhs + opPos + 1);

    int isFloat = isFloatOperand(y) || isFloatOperand(z);

    char opcode[10];
    if(op == '+')
        strcpy(opcode, "ADD");
    else if(op == '-')
        strcpy(opcode, "SUB");
    else if(op == '*')
        strcpy(opcode, "MUL");
    else if(op == '/')
        strcpy(opcode, "DIV");

    if(isFloat)
        strcat(opcode, "F");

    char movOp[10];
    strcpy(movOp, isFloat ? "MOVF" : "MOV");

    printf("\n%s ", movOp);
    printOperand(z);
    printf(",R1\n");

    printf("%s ", opcode);
    printOperand(y);
    printf(",R1\n");

    printf("%s R1,%s\n", movOp, x);

    return 0;
}
```

Output

```
Enter the Three Address Code : a=b+c;

MOV c,R1
ADD b,R1
MOV R1,a


=== Code Execution Successful ===
```

Output

```
Enter the Three Address Code : area=radius*2;

MOV #2,R1
MUL radius,R1
MOV R1,area


=== Code Execution Successful ===
```

Output

```
Enter the Three Address Code : a=26*22;

MOV #22,R1
MUL #26,R1
MOV R1,a


=== Code Execution Successful ===
```

Output

```
Enter the Three Address Code : ab=cd-10;

MOV #10,R1
SUB cd,R1
MOV R1,ab


=== Code Execution Successful ===
```

Output

```
Enter the Three Address Code : Islabudeen=Compiler+BKT0022;

MOV BKT0022,R1
ADD Compiler,R1
MOV R1,Islabudeen


=== Code Execution Successful ===
```

Output

```
Enter the Three Address Code : Test=3.14*r;

MOVF r,R1
MULF #3.14,R1
MOVF R1,Test


=== Code Execution Successful ===
```

Output

```
Enter the Three Address Code : t1=bhoomi*0130;

MOV #0130,R1
MUL bhoomi,R1
MOV R1,t1


=== Code Execution Successful ===
```



Result:

Thus, the program to translate a given Three-Address Code statement into Assembly Code was successfully developed. It correctly identifies the operator, distinguishes constants from identifiers, handles floating-point operands using the 'F'-suffixed opcodes and MOVF, and produces the corresponding Assembly Code instructions.